

# MetaOpticsAI

**Technical Documentation of the Physics Core**  
**Computational Nanophotonics for Metasurface Design**

MetaOpticsAI contributors

August 8, 2026

## Abstract

METAOPTICSAI is a pure-Python framework for the end-to-end design of metasurface-based flat optics. Starting from a user-level specification (wavelength, numerical aperture, aperture size, meta-atom material) the package generates a phase profile, sweeps meta-atom geometries with Rigorous Coupled-Wave Analysis (RCWA), analyses the far-field Point Spread Function (PSF), quantifies imaging quality via Modulation Transfer Function (MTF), Strehl ratio and Zernike decomposition, and finally exports a fabrication-ready GDSII file. This document derives the physical models that underpin each module, presents the concrete API, and shows how the pieces compose into a working design flow. The scope of this document is the physics core of METAOPTICSAI as it exists after the removal of the LLM/RAG/MCP orchestration layer; those components are preserved for reference in the repository's `old_one/` directory and are only briefly summarized in Appendix B.

# Contents

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction and Architecture</b>                            | <b>3</b>  |
| 1.1      | Why METAOPTICSAI . . . . .                                      | 3         |
| 1.2      | Package Layout . . . . .                                        | 3         |
| 1.3      | Notational Conventions . . . . .                                | 4         |
| <b>2</b> | <b>Installation and Configuration</b>                           | <b>5</b>  |
| 2.1      | Installation . . . . .                                          | 5         |
| 2.2      | Configuration — mod <i>configs/settings.py</i> . . . . .        | 5         |
| <b>3</b> | <b>Domain Types</b>                                             | <b>7</b>  |
| 3.1      | MetalensDesignParams . . . . .                                  | 7         |
| 3.2      | IncidenceSpec . . . . .                                         | 8         |
| 3.3      | Result Dataclasses . . . . .                                    | 8         |
| <b>4</b> | <b>Phase Profiles and Quantization</b>                          | <b>9</b>  |
| 4.1      | Fourier Optics Preliminaries . . . . .                          | 9         |
| 4.2      | Focusing Metalens (Fresnel Zone Lens) . . . . .                 | 9         |
| 4.3      | Axicon (Bessel-Beam Generator) . . . . .                        | 10        |
| 4.4      | Spiral Phase Plate (Vortex Beam) . . . . .                      | 10        |
| 4.5      | Loading a Phase Mask from an Image . . . . .                    | 10        |
| 4.6      | Quantization . . . . .                                          | 10        |
| <b>5</b> | <b>Rigorous Coupled-Wave Analysis</b>                           | <b>12</b> |
| 5.1      | Motivation . . . . .                                            | 12        |
| 5.2      | Maxwell’s Equations in Periodic Media . . . . .                 | 12        |
| 5.3      | Fourier Expansion of the Permittivity . . . . .                 | 12        |
| 5.4      | Layer Eigen-Problem . . . . .                                   | 13        |
| 5.5      | S-Matrix Layer Cascade . . . . .                                | 13        |
| 5.6      | The Backend Abstraction . . . . .                               | 13        |
| 5.7      | The High-Level Facade: <code>sweep_diameter</code> . . . . .    | 14        |
| <b>6</b> | <b>Materials and Dispersion</b>                                 | <b>15</b> |
| <b>7</b> | <b>The Built-In FDTD Reference Library</b>                      | <b>16</b> |
| <b>8</b> | <b>Field Propagation</b>                                        | <b>17</b> |
| 8.1      | Scalar Diffraction Theory . . . . .                             | 17        |
| 8.1.1    | Angular Spectrum Method (Exact for Homogeneous Media) . . . . . | 17        |
| 8.1.2    | Fresnel and Fraunhofer Regimes . . . . .                        | 17        |
| 8.2      | The <code>propagate</code> Facade . . . . .                     | 18        |

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| <b>9</b>  | <b>Analysis Toolkit</b>                           | <b>19</b> |
| 9.1       | Point Spread Function . . . . .                   | 19        |
| 9.2       | Modulation Transfer Function . . . . .            | 19        |
| 9.3       | Strehl Ratio . . . . .                            | 20        |
| 9.4       | Zernike Decomposition . . . . .                   | 20        |
| 9.5       | Encircled Energy and Far-Field . . . . .          | 21        |
| <b>10</b> | <b>Optimization</b>                               | <b>22</b> |
| 10.1      | Optimizer Base Class . . . . .                    | 22        |
| 10.2      | Heuristic Optimizer . . . . .                     | 22        |
| 10.3      | Gradient Optimizer . . . . .                      | 23        |
| 10.4      | Surrogate Optimizer . . . . .                     | 23        |
| <b>11</b> | <b>Fabrication and GDS Export</b>                 | <b>24</b> |
| 11.1      | From Quantized Mask to GDSII . . . . .            | 24        |
| 11.2      | Geometry Primitives . . . . .                     | 24        |
| <b>12</b> | <b>The Tool Layer</b>                             | <b>26</b> |
| 12.1      | Motivation . . . . .                              | 26        |
| 12.2      | The <code>ArtifactStore</code> Protocol . . . . . | 26        |
| 12.3      | Composing a Registry . . . . .                    | 27        |
| 12.4      | Available Tools . . . . .                         | 27        |
| <b>A</b>  | <b>Full End-to-End Example</b>                    | <b>28</b> |
| <b>B</b>  | <b>Legacy AI Layer (<code>old_one/</code>)</b>    | <b>30</b> |

# Chapter 1

## Introduction and Architecture

### 1.1 Why MetaOpticsAI

Classical refractive optics rely on the accumulation of optical path length through bulk dielectric to shape a wavefront. A *metasurface* replaces this bulk with a sub-wavelength array of scatterers—the “meta-atoms”—each of which imprints a locally-defined phase on the transmitted (or reflected) field. By choosing the scatterer geometry pixel by pixel, an essentially arbitrary phase profile  $\varphi(x, y)$  can be printed in a single lithography step. The resulting flat optical elements are 1–3 orders of magnitude thinner than the diffraction-limited focal length they synthesize, opening the door to compact imaging systems, augmented-reality combiners, LiDAR beam steerers, and polarization-sensitive holograms yu2011,khorasaninejad2016,capasso2018.

Designing such a device requires four physically-distinct calculations executed in sequence:

1. **Phase-profile synthesis** Turn the imaging spec (focal length, aperture, wavelength) into a continuous target phase  $\varphi(x, y)$ .
2. **Unit-cell electromagnetic simulation** For each candidate meta-atom geometry, solve Maxwell’s equations in a periodic unit cell to obtain the complex transmission  $t = Ae^{i\phi}$  that the meta-atom imposes on the incident field.
3. **Phase-to-geometry mapping and quantization** Invert the unit-cell library to select, at every pixel, the meta-atom whose  $\phi$  is closest to  $\varphi(x, y)$ ; discretize the pixel grid to fabricable dimensions.
4. **Far-field analysis and layout** Propagate the emitted field, compute the PSF/MTF/Strehl, decide whether the design meets spec, and if so emit a GDS layout.

Each of these four stages has an entire textbook worth of theory behind it. METAOPTICS\_AI provides a small, opinionated implementation of each and glues them together through a set of typed data classes so that a design engineer can move from user spec to `.gds` in tens of lines of Python.

### 1.2 Package Layout

The source tree lives under `mod src/metaopticsai/`. Every subpackage listed below is documented in a dedicated `ch`

| Subpackage                    | Purpose                                             | Chapter |
|-------------------------------|-----------------------------------------------------|---------|
| <code>mod configs</code>      | Pydantic settings, filesystem paths                 | §2      |
| <code>mod domain</code>       | Pure data types (design params, results, materials) | §3      |
| <code>mod phase</code>        | Analytical phase profiles + quantization            | §4      |
| <code>mod physics</code>      | RCWA, FDTD library, propagation, backends           | §5–8    |
| <code>mod analysis</code>     | PSF, MTF, Strehl, Zernike, encircled energy         | §9      |
| <code>mod optimization</code> | Heuristic / gradient / surrogate optimizers         | §10     |
| <code>mod fabrication</code>  | GDSII generation (via <code>gdsfactory</code> )     | §11     |
| <code>mod tools</code>        | Pydantic-schema tool wrappers around the above      | §12     |
| <code>mod utils</code>        | Logging, JSON I/O, async helpers, images            | —       |
| <code>mod vendor</code>       | Vendored <code>metabox3</code> (the RCWA engine)    | —       |

The single hard rule enforced by design is the direction of imports. `mod domain` depends only on the standard library. `mod phase`, `mod analysis` and `mod fabrication` depend on `mod domain` and `NumPy` but nothing else. `mod physics` depends on `mod domain` plus a chosen backend (vendored `metabox3` for the differentiable RCWA `Apax` matrix path, or the analytical stand-in). Nothing in `mod domain`, `mod phase`, `mod analysis`, `mod fabrication`, or `mod optimization` imports from a higher layer, which means each of these can be used independently.

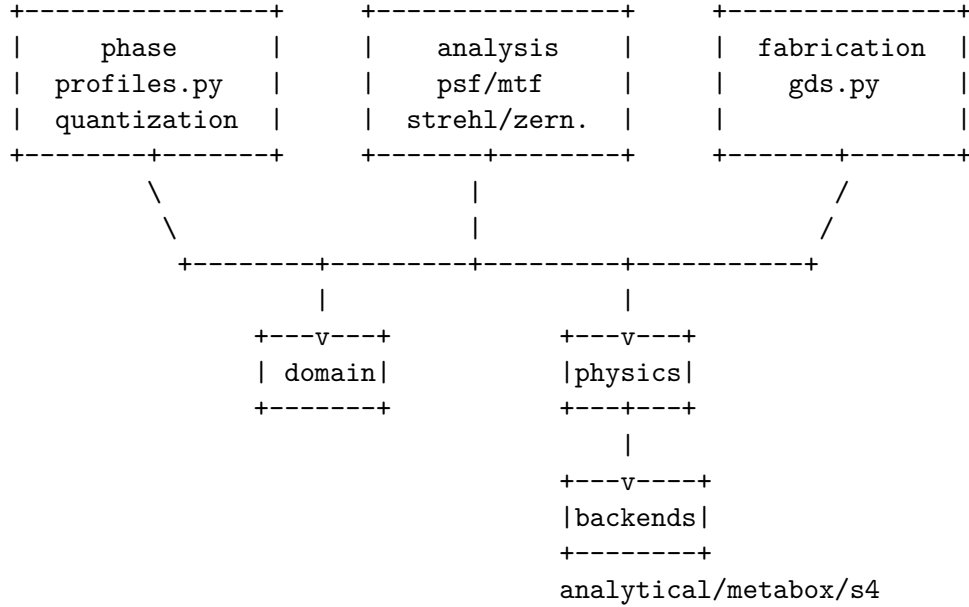


Figure 1.1: Layered architecture. Arrows point in the direction of the `import` relationship. No cycles; the `mod physics` layer is the only one that speaks to an EM backend.

### 1.3 Notational Conventions

Throughout this document, boldface symbols ( $\mathbf{E}$ ,  $\mathbf{H}$ ) denote vector fields; italics denote scalar quantities; hats ( $\hat{\mathbf{x}}$ ) denote unit vectors. Wavelengths in code are expressed in nanometres and distances in micrometres or millimetres; conversion to SI (metres) occurs only at the boundary between the `mod domain` and `mod physics` layers. Angles in code are always in degrees, in equations always in radians.

## Chapter 2

# Installation and Configuration

### 2.1 Installation

METAOPTICSAI is a standard `pyproject.toml`-based package. Clone the repository and install it in editable mode:

```
1 git clone <repository-url> metacode
2 cd metacode
3 python -m venv .venv
4 source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
5 pip install -e .
```

Listing 2.1: Install in editable mode.

The base dependencies are lightweight (NumPy, SciPy, Pydantic, Pillow). To enable the differentiable RCWA backend and the gradient-based optimizer, install TensorFlow:

```
1 pip install "tensorflow>=2.13,<2.16"
```

Listing 2.2: Optional TF path.

To enable GDS export, install `gdsfactory`:

```
1 pip install gdsfactory
```

Both TensorFlow and `gdsfactory` are imported lazily; the package loads and the analytical backend runs even without them installed.

### 2.2 Configuration — mod `configs/settings.py`

The single `Settings` singleton exposed at `metaopticsai.configs.settings.settings` is the only source of runtime configuration. Two sub-configs live under it:

- `RCWAConfig` Grid harmonics, resolution and mini-batch size for the vendored `metabox3` RCWA solver.
- `PathsConfig` Deterministic filesystem paths derived from the project root: `output_dir`, `papers_dir`, `material_dir`.

```
1 from metaopticsai.configs.settings import settings
2
3 print(settings.rcwa.harmonics)          # 3
4 print(settings.rcwa.resolution)         # 64
5 print(settings.paths.project_root)      # <repo>/
6 print(settings.paths.output_dir)        # <repo>/outputs/
```

```
7 print(settings.paths.material_dir)      # <repo>/src/.../vendor/metabo3/material_data
```

Listing 2.3: The full contents of `configs/settings.py`.

The RCWA fields respond to environment variables with the prefix `RCWA_`, e.g. `RCWA_HARMONICS=5`. A `.env` file at the repository root is loaded automatically via `pydantic-settings`.

## Chapter 3

# Domain Types

The `mod domainsubpackage` is the vocabulary of the package. Every function in every other layer either accepts or

### 3.1 MetalensDesignParams

`MetalensDesignParams` encodes the complete specification of a metalens. Its fields fall into four groups: the optical axis (wavelength, focal length, diameter), the meta-atom geometry (material, pillar height, min/max diameters, periodicity), the RCWA sampling (harmonics, radial pixels, substrate index), and the polarization (Jones vector). Field constraints (`ge`, `gt`, `lt`) are enforced by Pydantic V2 at construction, and a cross-field validator ensures that  $d_{\min} < d_{\max}$  and  $d_{\max} < P - 20$  nm so pillars fit inside the unit cell with a 20 nm wall margin.

```
1 from metaopticsai.domain.design import MetalensDesignParams
2
3 params = MetalensDesignParams(
4     wavelength_nm=532,
5     focal_length_mm=2.0,
6     diameter_mm=1.0,
7     pillar_material="TiO2",
8     pillar_height_nm=600,
9     min_diameter_nm=80,
10    max_diameter_nm=300,
11    periodicity_nm=350,
12 )
13 print(params.numerical_aperture) # 0.243
14 print(params.f_number)          # 2.0
15 print(params.aspect_ratio)      # 3.16
```

Listing 3.1: Building a design spec with validated fields.

Three computed fields simplify downstream code:

$$\text{NA} = \frac{D/2}{\sqrt{(D/2)^2 + f^2}}, \quad (3.1)$$

$$f/\# = \frac{f}{D}, \quad (3.2)$$

$$\bar{d} = \frac{1}{2}(d_{\min} + d_{\max}), \quad (3.3)$$

with  $D$  the aperture diameter and  $f$  the focal length.

## 3.2 IncidenceSpec

`IncidenceSpec` captures the illumination: one or more wavelengths (in metres for backend consumption),  $(\theta, \phi)$  angles of incidence, and a Jones vector. Its only convenience constructor, `at_wavelength_nm`, is a shorthand for the common on-axis case:

```
1 from metaopticsai.domain.incidence import IncidenceSpec
2
3 inc = IncidenceSpec.at_wavelength_nm(532)
4 # same as: IncidenceSpec(wavelength_m=[532e-9])
```

## 3.3 Result Dataclasses

The `mod domain/results.py` module defines a small set of frozen dataclasses that flow upward from a physics back-

- `UnitCellResult` — complex transmission of one meta-atom.
- `RCWSweepResult` — amplitudes and phases along one swept parameter (typically diameter).
- `FieldResult` —  $|E(x, y, z)|^2$  after propagation.
- `PSFResult` — PSF plus derived Strehl ratio and FWHM.
- `FOMResult` — one scalar figure of merit with provenance.

Each class exposes a `to_payload()` method that renders NumPy arrays into JSON-serializable lists so they can cross the tool boundary without custom encoders.

## Chapter 4

# Phase Profiles and Quantization

### 4.1 Fourier Optics Preliminaries

A metasurface at  $z = 0$  is fully characterised, in the thin-element approximation, by a complex transmission function

$$t(x, y) = A(x, y) e^{i\varphi(x, y)}, \quad (4.1)$$

with  $A \in [0, 1]$  the local amplitude transmittance and  $\varphi \in [0, 2\pi)$  the local phase. Under paraxial illumination by a plane wave  $U_{\text{in}} = A_0$ , the field just after the surface is  $U(x, y, 0^+) = A_0 t(x, y)$ . Downstream propagation is treated in Chapter 8; here we focus on synthesising  $\varphi$ .

### 4.2 Focusing Metalens (Fresnel Zone Lens)

To collapse an incident plane wave into a point at  $(0, 0, f)$ , geometric optics demands that every ray leaving the lens carry the same optical path length to the focal point. For a ray leaving the lens at radial coordinate  $r = \sqrt{x^2 + y^2}$ , the free-space distance to focus is  $\sqrt{r^2 + f^2}$ , whereas the axial ray travels only  $f$ . The extra optical path  $\sqrt{r^2 + f^2} - f$  must therefore be *subtracted* by the surface:

$$\varphi_{\text{FZL}}(r) = 2\pi - \frac{2\pi}{\lambda} \left( \sqrt{r^2 + f^2} - f \right) \bmod 2\pi. \quad (4.2)$$

The leading  $2\pi$  term is a global piston and has no physical effect; it is retained in the code to keep the phase strictly non-negative before wrapping. Equation (4.2) is the hyperbolic phase of a diffractive Fresnel Zone Lens (FZL); its *unwrapped* form is quadratic near the axis and diverges as  $r \rightarrow \infty$ , which is why we always wrap modulo  $2\pi$  before quantization.

```
1 from metaopticsai.phase.profiles import fresnel_zone_lens
2
3 phase = fresnel_zone_lens(
4     wavelength_nm=532,
5     focal_length_um=2000,      # f = 2 mm
6     diameter_um=500,          # D = 0.5 mm
7     pixel_size_um=0.5,
8     circular=True,
9 )
10 print(phase.shape, phase.min(), phase.max()) # (2000,2000) 0.0 ~2*pi
```

Listing 4.1: Generating an FZL phase mask.

The output is a square array of side  $N = 2[R/p]$  (with  $R$  the aperture radius and  $p$  the pixel size). Points outside the circular aperture are set to zero when `circular=True`; this zero-mask is what the `compute_strehl_ratio` function later uses to detect the aperture support.

*Remark 4.1.* Numerical aperture is bounded by  $\text{NA} = R/\sqrt{R^2 + f^2}$ ; a fair grid should over-sample this by at least  $2\times$  to avoid aliasing at the lens edge, i.e.  $p \lesssim \lambda/(2\text{NA})$ .

### 4.3 Axicon (Bessel-Beam Generator)

An axicon imparts a conical phase ramp that transforms an incident Gaussian into a Bessel-like beam over a diffraction-free zone. The required phase, derived by unwrapping the ray-tracing argument along the cone with half-angle  $\alpha$ , is

$$\varphi_{\text{axicon}}(r) = k(R - r) \tan \alpha \bmod 2\pi, \quad (4.3)$$

with  $k = 2\pi/\lambda$  the wavenumber. The Bessel zone has axial extent

$$z_{\text{max}} = \frac{R}{\tan \alpha}. \quad (4.4)$$

```
1 from metaopticsai.phase.profiles import axicon
2
3 phi = axicon(wavelength_nm=633, cone_angle_deg=1.0,
4             diameter_um=1000, pixel_size_um=1.0)
```

### 4.4 Spiral Phase Plate (Vortex Beam)

The vortex, or Laguerre–Gauss  $\text{LG}_0^\ell$ , mode carries orbital angular momentum  $\ell\hbar$  per photon. It is generated by a helical phase in the azimuthal angle:

$$\varphi_{\text{SPP}}(\theta) = \ell\theta \bmod 2\pi. \quad (4.5)$$

Non-integer  $\ell$  is possible but produces fractional vortices with opening branch cuts.

```
1 from metaopticsai.phase.profiles import spiral_phase_plate
2
3 phi = spiral_phase_plate(charge=2, diameter_um=500, pixel_size_um=0.5)
```

### 4.5 Loading a Phase Mask from an Image

For fully-custom targets (e.g. holograms, arbitrary CGH), an 8-bit grayscale image can be interpreted as  $[0, 255] \mapsto [0, 2\pi)$ :

```
1 from metaopticsai.phase.profiles import from_image
2 phi = from_image("target.png")
```

### 4.6 Quantization

A continuous phase mask must be reduced to a small number of levels before it can be mapped to a discrete meta-atom library. Standard choices are  $L = 4$  (2-bit),  $L = 8$  (3-bit) or  $L = 16$  (4-bit). Given a target range  $[\varphi_{\min}, \varphi_{\max}]$ , the quantizer defines equally-spaced bin edges

$$b_i = \varphi_{\min} + i\Delta, \quad \Delta = \frac{\varphi_{\max} - \varphi_{\min}}{L}, \quad i = 0, \dots, L - 1, \quad (4.6)$$

and returns an integer level  $\ell_{ij} \in \{1, \dots, L\}$  per pixel by `numpy.digitize`. Because meta-atom libraries are almost always tabulated in degrees, the API accepts `phase_min_deg` and `phase_max_deg` in degrees:

```

1 import numpy as np
2 from metaopticsai.phase.quantization import quantize
3
4 quant, bins = quantize(phase, n_levels=8)      # phase in radians
5 print(quant.dtype, quant.min(), quant.max())  # int, 1, 8
6 print(bins)                                   # [0, 45, 90, ... , 315]

```

Listing 4.2: From continuous  $\varphi$  to fabricable levels.

The Strehl loss due to  $L$ -level quantization is bounded above by swanson1991:

$$S \geq 2^{\frac{\pi}{L}}, \quad (4.7)$$

which yields  $S \geq 0.81$  for  $L = 4$  and  $S \geq 0.95$  for  $L = 8$ .

## Chapter 5

# Rigorous Coupled-Wave Analysis

### 5.1 Motivation

The transmission  $t(x, y)$  in Eq. (4.1) is not a free parameter that the designer picks; it is the outcome of a full-vector solution of Maxwell's equations in the actual meta-atom geometry. For a periodic (or locally periodic) array of pillars, the tool of choice is *Rigorous Coupled-Wave Analysis* (RCWA), also known in the literature as the Fourier Modal Method (FMM) moharam1995,liChandezon2013. The central idea: expand the fields into Bloch harmonics, expand the permittivity into a Fourier series, and match tangential  $\mathbf{E}$  and  $\mathbf{H}$  at every layer interface via an S-matrix.

### 5.2 Maxwell's Equations in Periodic Media

In a source-free, non-magnetic, linear medium at frequency  $\omega$ , the time-harmonic Maxwell equations reduce to

$$\nabla \times \mathbf{E} = i\omega\mu_0\mathbf{H}, \quad (5.1)$$

$$\nabla \times \mathbf{H} = -i\omega\varepsilon_0\varepsilon_r(\mathbf{r})\mathbf{E}. \quad (5.2)$$

Consider a unit cell of periodicities  $\Lambda_x, \Lambda_y$  in the  $(x, y)$  plane, stacked into  $L$  layers each of uniform  $z$ -thickness  $h_\ell$  but with  $z$ -independent,  $(x, y)$ -periodic  $\varepsilon_r^{(\ell)}(x, y)$ . By the Bloch–Floquet theorem, the fields inside such a medium can be written

$$\mathbf{E}(x, y, z) = \sum_{m,n} \mathbf{E}_{mn}(z) e^{i(k_{x,m}x + k_{y,n}y)}, \quad (5.3)$$

with transverse wavevectors

$$k_{x,m} = k_{x,0} + \frac{2\pi m}{\Lambda_x}, \quad k_{y,n} = k_{y,0} + \frac{2\pi n}{\Lambda_y}, \quad (5.4)$$

and  $(k_{x,0}, k_{y,0})$  set by the illumination angle  $(\theta, \phi)$  through  $k_{x,0} = k_0 \sin \theta \cos \phi$ ,  $k_{y,0} = k_0 \sin \theta \sin \phi$ ,  $k_0 = \omega/c$ .

### 5.3 Fourier Expansion of the Permittivity

Inside layer  $\ell$  the permittivity is expanded as

$$\varepsilon_r^{(\ell)}(x, y) = \sum_{p,q} \hat{\varepsilon}_{pq}^{(\ell)} e^{i(2\pi p/\Lambda_x)x + i(2\pi q/\Lambda_y)y}, \quad (5.5)$$

with coefficients obtained from the discrete Fourier transform of the sampled unit cell. Truncating the series at  $|m|, |p| \leq N_x$  and  $|n|, |q| \leq N_y$  produces a finite Toeplitz matrix  $[\varepsilon^{(\ell)}]$  of size  $(2N_x + 1)(2N_y + 1)$ . Careful ordering of  $\varepsilon$  vs.  $1/\varepsilon$  series (the “inverse rule” of Li li1996) is required for TM modes; the vendored `metabox3` implements it.

## 5.4 Layer Eigen-Problem

Substituting the expansions into (5.2) yields, inside each layer, an eigenvalue problem for the  $z$ -propagation constants  $\gamma_p^{(\ell)}$ :

$$[M^{(\ell)}] \mathbf{s}_p = \gamma_p^2 \mathbf{s}_p, \quad (5.6)$$

where  $M^{(\ell)}$  is the  $2M \times 2M$  transverse Maxwell operator ( $M = (2N_x + 1)(2N_y + 1)$ ) constructed from the Toeplitz permittivity and the diagonal  $k$ -vector matrices. Each eigen-mode propagates as  $\exp(\pm \gamma_p z)$  inside its layer.

## 5.5 S-Matrix Layer Cascade

The full multilayer response is obtained by cascading single-layer scattering matrices  $S^{(\ell)}$  using the numerically stable Redheffer star product redheffer1959:

$$S^{(\text{tot})} = S^{(1)} \star S^{(2)} \star \dots \star S^{(L)}. \quad (5.7)$$

The transmission harmonics extracted from the top-right block of  $S^{(\text{tot})}$ , after projection onto the desired polarization, yield the complex transmission  $t_{mn}$ . For a first-order approximation of the metasurface phase, one keeps only the zeroth-order transmission,

$$t \approx t_{00} = A e^{i\phi}, \quad (5.8)$$

which is exactly the object returned by `UnitCellResult` in Chapter 3.

## 5.6 The Backend Abstraction

METAOPTICSAI treats the RCWA solver as a pluggable backend. Three backends ship with the package:

- **AnalyticalBackend** — no external dependency; uses closed-form heuristics for  $A(d)$  and  $\phi(d)$  derived from Fresnel reflection at the pillar cap. Always available; used for CI and quick iteration.
- **MetaboxBackend** — differentiable TensorFlow-based RCWA (vendored under `mod vendor/metabox3`). *Enab*
- **S4Backend** — placeholder for Stanford S4 (currently disabled).

They all implement the same `SimulationBackend` interface:

```

1 class SimulationBackend(ABC):
2     name: str
3     def is_available(self) -> bool: ...
4     def simulate_unit_cell(self, params, incidence) -> UnitCellResult: ...
5     def sweep_unit_cells(self, params, incidence,
6                           sweep_param, sweep_values) -> RCWASweepResult: ...
7     def propagate_field(self, phase_mask, wavelength_m, pixel_size_m,
8                           distance_m, refractive_index=1.0) -> FieldResult: ...

```

Listing 5.1: The `SimulationBackend` contract.

A backend is resolved through the `BackendRegistry`. Its `get("auto")` method returns the first available backend in the priority order `metabox`  $\rightarrow$  `s4`  $\rightarrow$  `analytical`. Tests inject a custom registry via `set_registry()`.

## 5.7 The High-Level Facade: `sweep_diameter`

The most common RCWA call in a design flow is a diameter sweep, which produces a phase-vs-diameter curve used later to invert  $\varphi(x, y) \mapsto d(x, y)$ :

```

1 from metaopticsai.domain.design import MetalensDesignParams
2 from metaopticsai.physics.rcwa import sweep_diameter
3
4 params = MetalensDesignParams(wavelength_nm=532,
5     diameter_mm=1.0, focal_length_mm=2.0,
6     min_diameter_nm=80, max_diameter_nm=300, periodicity_nm=350)
7
8 result = sweep_diameter(params, wavelength_nm=532, n=20)
9 print(result.backend)           # "analytical" or "metabox"
10 print(result.phase_coverage)    # in [0, 1]; want >= 0.95 for a good library
11 print(result.mean_transmission) # want > 0.7

```

Listing 5.2: End-to-end diameter sweep with automatic backend selection.

`RCWASweepResult` carries the raw arrays `sweep_values`, `amplitudes`, `phases_rad`, plus two derived scalars: *phase coverage* (fraction of  $2\pi$  spanned) and *mean transmission*. These two scalars fully determine the Strehl-ratio estimate used by `HeuristicOptimizer` in Chapter 10.

## Chapter 6

# Materials and Dispersion

The vendored `metabox3` ships with tabulated  $n(\lambda) + ik(\lambda)$  curves for the standard nanophotonic materials as CSV files under `vendor/metabox3/material_data/`. METAOPTICSAI exposes them through a thin, backend-agnostic API in `mod physics/materials.py`.

```
1 from metaopticsai.physics.materials import list_materials, get_material_index
2
3 print(list_materials())           # ['Al', 'GaN', 'Si', 'SiN', 'TiO2', ...]
4
5 n_TiO2 = get_material_index("TiO2", wavelength_nm=532)
6 print(n_TiO2.n, n_TiO2.k)       # 2.55, ~0
7 print(n_TiO2.valid_range_um)    # (0.36, 0.79)
```

`get_material_index` performs a cubic interpolation on the tabulated data; it raises `ValueError` if the requested wavelength is outside the CSV's tabulated range. Physically, the interpolation should be regarded as reliable only well within that range.

**Example 6.1** (Dispersion of a broadband design). A broadband achromat requires materials whose refractive index changes slowly with  $\lambda$ .  $\text{TiO}_2$  has  $\Delta n \approx 0.15$  from 400 to 800 nm, which limits the achievable achromatic bandwidth.

## Chapter 7

# The Built-In FDTD Reference Library

Full-vector Finite-Difference Time-Domain (FDTD) simulations are expensive enough that pre-computed phase-vs-dimension curves for canonical meta-atom shapes are still the fastest path to a working design. The module `mod physics/fdtd.py` ships *fivesuchcurves*, *tabulatedforTiO<sub>2</sub>* fins and silicon nanocylinders at 405, 532, 633, 850 and 1064 nm.

```
1 from metaopticsai.physics.fdtd import FDTDStore, interpolate_phase_to_dimension
2
3 store = FDTDStore()
4 print(store.wavelengths())           # [405, 532, 633, 850, 1064]
5
6 entry = store.get(532)
7 print(entry.shape, entry.material)   # "Fin", "TiO2"
8 print(entry.height, entry.period)   # 600, 325
9
10 dims_fine, phases_fine, f_interp = interpolate_phase_to_dimension(
11     entry.dimensions, entry.phases,
12 )
```

Listing 7.1: Retrieving a built-in FDTD entry.

A user can also load an Excel table (`.xls` or `.xlsx`) of  $(d, \phi)$  pairs measured or FDTD-simulated in an external tool:

```
1 store.load_excel(
2     "my_lumerical_export.xlsx",
3     wl=780, shape="Cylinder", material="Si",
4     height=280, period=250,
5 )
```

The interpolator `interpolate_phase_to_dimension` returns the callable used by the fabrication engine (Chapter 11) to invert  $\phi \rightarrow d$  at every pixel.

## Chapter 8

# Field Propagation

### 8.1 Scalar Diffraction Theory

Once we have the field at the exit surface of the metalens,  $U(x, y, 0^+)$ , the intensity anywhere in the half-space  $z > 0$  is obtained by solving the Helmholtz equation with an outgoing-wave boundary condition. Under the scalar approximation (valid because meta-atoms produce fields that are locally paraxial for the free-space propagation), the exact solution is the Rayleigh–Sommerfeld diffraction integral gbur2011:

$$U(x, y, z) = \frac{1}{i\lambda} \iint U(x', y', 0^+) \frac{e^{ikR}}{R} \cos \theta \, dx' dy', \quad (8.1)$$

with  $R = \sqrt{(x - x')^2 + (y - y')^2 + z^2}$  and  $\theta$  the angle between  $\hat{\mathbf{R}}$  and  $\hat{\mathbf{z}}$ . Equation (8.1) is exact but expensive; several approximations are useful.

#### 8.1.1 Angular Spectrum Method (Exact for Homogeneous Media)

Take the 2-D Fourier transform of the field at  $z = 0^+$ :

$$\tilde{U}(k_x, k_y, 0^+) = \iint U(x, y, 0^+) e^{-i(k_x x + k_y y)} dx dy. \quad (8.2)$$

Each plane-wave component propagates independently with  $z$ -wavenumber  $k_z = \sqrt{k^2 - k_x^2 - k_y^2}$  (evanescent when  $k_x^2 + k_y^2 > k^2$ ). Recombine:

$$U(x, y, z) = \mathcal{F}^{-1} \left\{ \tilde{U}(k_x, k_y, 0^+) e^{ik_z z} \right\} (x, y). \quad (8.3)$$

This is what `propagate` calls; the metabox backend uses a tensor-friendly FFT, and the analytical backend falls back to Fraunhofer (large- $z$ ) when the propagator matrix would exceed memory.

#### 8.1.2 Fresnel and Fraunhofer Regimes

Expanding  $k_z$  to second order in  $(k_x, k_y)$  recovers the Fresnel propagator; further neglecting the quadratic term inside the exit aperture yields Fraunhofer far-field diffraction:

$$U(x, y, z)|_{\text{Fraunhofer}} \propto \mathcal{F}\{U(x', y', 0^+)\}|_{k_x=kx/z, k_y=ky/z}. \quad (8.4)$$

Fraunhofer is the regime used by `compute_psf` for the far-field PSF.

## 8.2 The propagate Facade

```
1 from metaopticsai.physics.propagation import propagate
2
3 field = propagate(
4     phase_mask=phi,           # 2D numpy array in radians
5     wavelength_nm=532,
6     pixel_size_um=0.5,
7     distance_um=2000,        # z = 2 mm
8     refractive_index=1.0,
9 )
10 print(field.intensity_2d.shape) # matches phi.shape
11 print(field.peak_intensity)
```

Listing 8.1: Angular-spectrum propagation through the current backend.

## Chapter 9

# Analysis Toolkit

The `mod analysis` package quantifies imaging performance from a phase mask. It contains five orthogonal com

### 9.1 Point Spread Function

The Fraunhofer PSF of a coherently-illuminated aperture is the squared-magnitude Fourier transform of the exit-aperture field:

$$\text{PSF}(x, y) = \left| \mathcal{F}\{A(x', y') e^{i\varphi(x', y')}\}(x, y) \right|^2. \quad (9.1)$$

In discrete form, we pad the  $N \times N$  aperture by a factor  $p \geq 4$  to over-sample the Airy pattern, then do a 2-D FFT. The transverse coordinates scale as  $\lambda z / (Np)$  so the pixel spacing in the focal plane is

$$\Delta x_{\text{focal}} = \frac{\lambda z}{N_{\text{pad}} \Delta x_{\text{ap}}}. \quad (9.2)$$

Setting  $z = f$  makes the code return coordinates in the focal plane; if  $z = 0$  is passed, the returned coordinates are dimensionless (per-pixel).

```
1 from metaopticsai.analysis.psf import compute_psf
2
3 psf, x_um, y_um = compute_psf(
4     phase_mask=phi, wavelength_nm=532,
5     pixel_size_um=0.5, focal_length_um=2000, pad_factor=4,
6 )
```

### 9.2 Modulation Transfer Function

The MTF is the magnitude of the Optical Transfer Function, which is in turn the Fourier transform of the (incoherent) PSF [goodman2005](#):

$$\text{OTF}(f_x, f_y) = \mathcal{F}\{\text{PSF}(x, y)\}, \quad \text{MTF} = |\text{OTF}|. \quad (9.3)$$

Because the aperture is (approximately) circularly symmetric, the module returns the *radial average* of the 2-D MTF, plus the ideal diffraction-limited MTF of the same NA for reference:

$$\text{MTF}_{\text{diff}}(f/f_c) = \frac{2}{\pi} \left[ \arccos(f/f_c) - (f/f_c) \sqrt{1 - (f/f_c)^2} \right], \quad (9.4)$$

with cutoff frequency  $f_c = 2\text{NA}/\lambda$  (in line-pairs per unit length). A design is diffraction-limited iff its radial MTF sits on top of Eq. (9.4) across all spatial frequencies.

```

1 from metaopticsai.analysis.mtf import compute_mtf
2
3 freqs_lpmm, mtf_radial, mtf_diff = compute_mtf(
4     psf_2d=psf, pixel_size_um=0.5,
5     wavelength_nm=532, focal_length_um=2000,
6 )

```

### 9.3 Strehl Ratio

The Strehl ratio  $\mathcal{S}$  is the on-axis intensity of the actual PSF normalized to that of the diffraction-limited PSF from the same aperture:

$$\mathcal{S} = \frac{|U_{\text{aberr}}(0, 0, f)|^2}{|U_{\text{ideal}}(0, 0, f)|^2}. \quad (9.5)$$

For small phase errors of RMS wavefront  $\sigma_\varphi$  (in radians), Maréchal's approximation gives

$$\mathcal{S} \approx e^{-\sigma_\varphi^2}, \quad (9.6)$$

and the diffraction-limited criterion is  $\mathcal{S} \geq 0.8$  (equivalent to  $\sigma_\varphi \leq \lambda/14$  RMS OPD). `compute_strehl_ratio` computes  $\mathcal{S}$  directly from two FFTs (ideal aperture and aberrated aperture) rather than from a Zernike expansion, so it is exact in the sampled-aperture sense.

```

1 from metaopticsai.analysis.strehl import compute_strehl_ratio
2
3 strehl, psf_aberr, psf_ideal = compute_strehl_ratio(
4     phase_mask=phi, pad_factor=4,
5 )
6 print(f"Strehl = {strehl:.3f}")

```

### 9.4 Zernike Decomposition

Zernike polynomials  $\{Z_j\}$  are the orthogonal basis for functions on the unit disk that separates aberrations by physical name (piston, tilt, defocus, astigmatism, coma, spherical, ...). METAOPTICSAI uses the *Noll* indexing convention noll1976, which numbers the polynomials sequentially rather than by  $(n, m)$ . The decomposition returned by `zernike_decomposition` is

$$c_j = \frac{1}{|\Omega|} \iint_{\Omega} \varphi(\rho, \theta) Z_j(\rho, \theta) \rho d\rho d\theta, \quad (9.7)$$

with  $\Omega$  the unit disk and  $\rho \in [0, 1]$ . The radial polynomials are

$$R_n^m(\rho) = \sum_{s=0}^{(n-|m|)/2} \frac{(-1)^s (n-s)!}{s! \left(\frac{n+|m|}{2} - s\right)! \left(\frac{n-|m|}{2} - s\right)!} \rho^{n-2s}, \quad (9.8)$$

and the normalized Zernikes are

$$Z_j(\rho, \theta) = \begin{cases} \sqrt{n+1} R_n^0(\rho), & m = 0, \\ \sqrt{2(n+1)} R_n^{|m|}(\rho) \cos(m\theta), & m > 0, \\ \sqrt{2(n+1)} R_n^{|m|}(\rho) \sin(|m|\theta), & m < 0. \end{cases} \quad (9.9)$$

```

1 from metaopticsai.analysis.zernike import zernike_decomposition
2
3 coeffs, names = zernike_decomposition(phi, n_terms=15)
4 for j in sorted(coeffs, key=lambda k: abs(coeffs[k]), reverse=True)[:5]:
5     print(f"{j:2d} {names[j]:24s} {coeffs[j]:+.3f} rad")

```

| $j$ | Aberration             | $j$ | Aberration                |
|-----|------------------------|-----|---------------------------|
| 1   | Piston                 | 9   | Trefoil (vertical)        |
| 2   | Tilt (x)               | 10  | Trefoil (oblique)         |
| 3   | Tilt (y)               | 11  | Primary spherical         |
| 4   | Defocus                | 12  | Secondary astigmatism (v) |
| 5   | Astigmatism (oblique)  | 13  | Secondary astigmatism (o) |
| 6   | Astigmatism (vertical) | 14  | Secondary coma (v)        |
| 7   | Coma (vertical)        | 15  | Secondary coma (h)        |
| 8   | Coma (horizontal)      |     |                           |

Table 9.1: Noll indexing of the first 15 Zernike polynomials used by METAOPTICSAI.

## 9.5 Encircled Energy and Far-Field

Two additional analyses live under `mod analysis/encircled_energy.py` and `mod analysis/farfield.py`: *theencircled – energyfunction*  $E(r) = \iint_{\rho \leq r} \text{PSF} \, dA$ , useful for spot-size metrics like the 80%-encircled radius, and a helper that projects the angular spectrum onto a hemisphere for far-field visualisation. Both follow the same signature style as the PSF/MTF calls above.

# Chapter 10

## Optimization

### 10.1 Optimizer Base Class

Every optimizer in METAOPTICSAI subclasses the abstract `Optimizer` interface:

```
1 class Optimizer(ABC):
2     name: str
3     @abstractmethod
4     def optimize(self, params: MetalensDesignParams, *,
5                 target_fom: float, max_iters: int,
6                 **kwargs) -> tuple[MetalensDesignParams, FOMResult, list[float]]:
7         ...
```

The return is a triple: the refined `MetalensDesignParams`, the final `FOMResult`, and the history of scalar figure-of-merit values per iteration (useful for convergence plots).

### 10.2 Heuristic Optimizer

`HeuristicOptimizer` does not actually search; it evaluates a closed-form Strehl-ratio estimate given a pre-computed RCWA sweep:

$$\mathcal{S}_{\text{heur}} = \bar{A}^2 \min\left(1, \frac{\Delta\phi}{2\pi}\right), \quad (10.1)$$

with  $\bar{A}$  the mean transmission and  $\Delta\phi$  the phase coverage of the sweep. The reasoning: any missing phase coverage causes stray diffraction, which reduces on-axis intensity linearly, while amplitude loss enters as its square in intensity.

```
1 from metaopticsai.physics.rcwa import sweep_diameter
2 from metaopticsai.optimization.heuristic import HeuristicOptimizer
3
4 sweep = sweep_diameter(params, wavelength_nm=532, n=20)
5
6 opt = HeuristicOptimizer()
7 _, fom, history = opt.optimize(
8     params, target_fom=0.9, max_iters=1,
9     sweep_result=sweep,
10 )
11 print(fom.value, fom.name, fom.backend)
```

Listing 10.1: Two-step design: sweep, then heuristic FOM.

## 10.3 Gradient Optimizer

`GradientOptimizer` runs a full differentiable RCWA loop through the vendored `metabox3` library. It constructs a `LensAssembly`, marks the meta-atom diameter as a variable `Feature`, and optimizes with Adam under the log-Strehl figure of merit:

$$\mathcal{L} = -\log \mathcal{S}(\{d_i\}), \quad (10.2)$$

where  $\{d_i\}$  are the per-ring diameters. TensorFlow computes the gradient  $\frac{\partial \mathcal{L}}{\partial d_i}$  through the entire RCWA + Fourier propagation stack. The learning rate is expressed in metres because diameters live in SI inside `metabox3`.

```
1 from metaopticsai.physics.backends import default_registry
2 from metaopticsai.optimization.gradient import GradientOptimizer
3
4 opt = GradientOptimizer(backends=default_registry())
5 _, fom, history = opt.optimize(
6     params, target_fom=0.85, max_iters=30, learning_rate=2e-9,
7 )
8 print(f"Final Strehl = {fom.value:.3f} after {len(history)} steps")
```

Listing 10.2: Gradient run (TF required).

*Remark 10.1.* The gradient path is orders of magnitude slower per iteration than the heuristic but converges to a genuine local optimum of the true RCWA FOM. Typical practice: use `HeuristicOptimizer` to screen candidate parameter sets, then hand the survivors to `GradientOptimizer` for refinement.

## 10.4 Surrogate Optimizer

`SurrogateOptimizer ( mod optimization/surrogate.py)trainsasmallneuralsurrogateonRCWAsamplesand time.ItsuseisidenticaltoGradientOptimizerexceptthatthemetamodelpathlivesinsettings.paths.output_di`

## Chapter 11

# Fabrication and GDS Export

### 11.1 From Quantized Mask to GDSII

After phase synthesis, quantization, and unit-cell library selection, one obtains a triple  $(\ell_{ij}, b_i, d(\phi))$ : an integer level per pixel, the phase corresponding to each level, and a callable mapping phase back to meta-atom dimension. `GDSEngine` converts this triple into a GDSII layout by placing, at every pixel, a cell containing the selected shape at the appropriate dimension.

```
1 from metaopticsai.phase.profiles import fresnel_zone_lens
2 from metaopticsai.phase.quantization import quantize
3 from metaopticsai.physics.fDTD import FDTDStore
4 from metaopticsai.fabrication.gds import GDSEngine
5
6 phi = fresnel_zone_lens(wavelength_nm=532, focal_length_um=2000,
7                         diameter_um=500, pixel_size_um=0.5)
8 quant, bins = quantize(phi, n_levels=8)
9
10 entry = FDTDStore().get(532)
11 gds = GDSEngine()
12 info = gds.generate(
13     quantized_mask=quant,
14     bins=bins,
15     dimensions=entry.dimensions,
16     phases=entry.phases,
17     period=entry.period,
18     pixel_size_um=0.5,
19     unitcells_per_pixel=1,
20     shape=entry.shape,
21     is_circular=True,
22     output_path="outputs/metalens_532nm.gds",
23     fin_width=entry.fin_width or 0,
24     fin_length=entry.fin_length or 0,
25 )
26 print(info) # {'time_seconds': ..., 'file_size_kb': ..., 'filepath': ...}
```

Listing 11.1: Complete GDS export.

### 11.2 Geometry Primitives

Three canonical scatterer shapes are supported:

- **Cylinder** Circular disc of radius  $d/2$ . Used for silicon nanodisks.
- **Cross** Twelve-vertex Greek cross of arm-length  $d$  and arm-width  $w$ . Used for polarization-insensitive designs.

- **Fin** Rotated rectangle of length  $\ell$ , width  $w$  and orientation  $\theta$ . Used for Pancharatnam-Berry (geometric) phase; the phase is imposed by rotating the fin rather than by changing its dimension.

Internally **GDSEngine** caches one **gdsfactory Component** per unique dimension so the final layout is a shallow hierarchy of references rather than millions of independent polygons. This is the difference between a 10 MB file and a 10 GB one for a  $1\text{ cm}^2$  lens.

## Chapter 12

# The Tool Layer

### 12.1 Motivation

The physics APIs in the preceding chapters are the low-level primitives. For programmatic control by higher-level orchestration (whether by an external workflow engine, a test harness, or a notebook), METAOPTICSAI wraps every primitive in a `BaseTool` subclass carrying a Pydantic Input schema, an optional Output schema, and a callable `_run` method.

```
1 class BaseTool(ABC, Generic[InputT, OutputT]):
2     name: str
3     description: str
4     Input: type[BaseModel]
5     Output: type[BaseModel] | None = None
6     needs_store: bool = False
7
8     def __init__(self, store: ArtifactStore | None = None):
9         if self.needs_store and store is None:
10             raise ValueError(f"{self.name} requires an ArtifactStore.")
11         self.store = store
12
13     async def call(self, raw_input): ...           # validate, run, validate
14     def call_sync(self, raw_input): ...           # blocking wrapper
15     @abstractmethod
16     def _run(self, params): ...
```

### 12.2 The ArtifactStore Protocol

Handles (opaque strings) are the currency between tools: a tool that produces a large NumPy array puts it in a store keyed by a UUID and returns just the handle, so the next tool can retrieve the payload without serialising it through the caller. The tool layer only requires that the injected store satisfy this small Protocol:

```
1 class ArtifactStore(Protocol):
2     def get(self, handle: str, *, expected_kind: str | None = ...) -> Any: ...
3     def put(self, kind: str, payload: Any, **metadata: Any) -> str: ...
4     def list(self, kind: str | None = ...) -> list: ...
```

Any object exposing these three methods will do; callers are free to supply an in-memory dict, a Redis-backed store, or a database. The concrete implementation once bundled under `mod store/wasmovedtoold_one/store/whentheAllayerwasretired; aminimalin-memoryversioncanbere-implementedin $\lesssim$  30 lines.`

## 12.3 Composing a Registry

`build_tool_registry(store, backends, job_manager)` is the single composition root. It lazy-imports every concrete tool class, instantiates each with the injected store and backends, and returns a `ToolRegistry` keyed by tool name.

```

1 from metaopticsai.physics.backends import default_registry
2 from metaopticsai.tools import build_tool_registry
3
4 # 1. Provide a minimal in-memory store (see notebook 04 in old_one/ for
5 #   the full one).
6 class DictStore:
7     def __init__(self): self._d = {}
8     def put(self, kind, payload, **meta):
9         import uuid; h = f"{kind}-{uuid.uuid4().hex[:8]}"
10        self._d[h] = (kind, payload, meta); return h
11    def get(self, handle, *, expected_kind=None):
12        kind, payload, meta = self._d[handle]
13        assert expected_kind is None or kind == expected_kind
14        class A:
15            def __init__(s): s.payload = payload; s.metadata = meta
16            return A()
17    def list(self, kind=None):
18        return [h for h, (k, _, _) in self._d.items() if kind in (None, k)]
19
20 registry = build_tool_registry(DictStore(), default_registry())
21 print(registry.names())           # ['get_fDTD_library', 'run_rcwa_sweep', ...]
22
23 sweep_tool = registry.get("run_rcwa_sweep")
24 result = await sweep_tool.call({
25     "params": params.to_dict(), "wavelength_nm": 532, "n": 20,
26 })

```

Listing 12.1: Programmatic use of the tool layer.

## 12.4 Available Tools

| Name                                  | Purpose                                        |
|---------------------------------------|------------------------------------------------|
| <code>get_fDTD_library</code>         | Introspect the built-in FDTD reference library |
| <code>run_rcwa_sweep</code>           | Parametric RCWA sweep                          |
| <code>list_materials</code>           | List CSV-tabulated materials                   |
| <code>get_material_index</code>       | $n + ik$ at a wavelength                       |
| <code>generate_phase_mask</code>      | Analytical phase profile generator             |
| <code>analyze_psf</code>              | Focal-plane PSF from a phase mask              |
| <code>analyze_mtf</code>              | Radial MTF of a PSF                            |
| <code>zernike_decompose</code>        | Zernike coefficients of a phase mask           |
| <code>propagate_field</code>          | Angular-spectrum propagation                   |
| <code>compute_max_intensity</code>    | Peak intensity FOM                             |
| <code>compute_center_intensity</code> | On-axis intensity FOM                          |
| <code>compute_mtf_volume</code>       | Broadband MTF integral                         |
| <code>export_gds</code>               | GDSII export via gdsfactory                    |
| <code>optimize_metallens</code>       | Heuristic/gradient optimizer                   |
| <code>train_metamodel</code>          | (Heavy job) surrogate training                 |
| <code>get_job</code>                  | Poll a submitted background job                |
| <code>list_artifacts</code>           | Enumerate handles in the store                 |

# Appendix A

## Full End-to-End Example

Putting all seven chapters together into a single script:

```
1 from metaopticsai.domain.design import MetalensDesignParams
2 from metaopticsai.physics.rcwa import sweep_diameter
3 from metaopticsai.phase.profiles import fresnel_zone_lens
4 from metaopticsai.phase.quantization import quantize
5 from metaopticsai.analysis.psf import compute_psf
6 from metaopticsai.analysis.mtf import compute_mtf
7 from metaopticsai.analysis.strehl import compute_strehl_ratio
8 from metaopticsai.analysis.zernike import zernike_decomposition
9 from metaopticsai.physics.fDTD import FDTDStore
10 from metaopticsai.fabrication.gds import GDSEngine
11
12 # --- 1. Design spec -----
13 params = MetalensDesignParams(
14     wavelength_nm=532, diameter_mm=1.0, focal_length_mm=2.0,
15     pillar_material="TiO2", pillar_height_nm=600,
16     min_diameter_nm=80, max_diameter_nm=300, periodicity_nm=350,
17 )
18
19 # --- 2. Meta-atom library (RCWA sweep) -----
20 sweep = sweep_diameter(params, wavelength_nm=532, n=20)
21 assert sweep.phase_coverage > 0.9, "Insufficient phase coverage"
22
23 # --- 3. Phase profile -----
24 phi = fresnel_zone_lens(
25     wavelength_nm=params.wavelength_nm,
26     focal_length_um=params.focal_length_mm * 1000,
27     diameter_um=params.diameter_mm * 1000,
28     pixel_size_um=0.5,
29 )
30
31 # --- 4. Analysis -----
32 strehl, *_ = compute_strehl_ratio(phi)
33 psf, _, _ = compute_psf(phi, params.wavelength_nm, pixel_size_um=0.5,
34                         focal_length_um=params.focal_length_mm * 1000)
35 freqs, mtf, mtf_ideal = compute_mtf(psf, pixel_size_um=0.5,
36                                     wavelength_nm=params.wavelength_nm,
37                                     focal_length_um=params.focal_length_mm * 1000)
38 coeffs, names = zernike_decomposition(phi, n_terms=15)
39 print(f"Strehl = {strehl:.3f}, dominant Zernike = "
40       f"{max(coeffs, key=lambda k: abs(coeffs[k]))}")
41
42 # --- 5. Fabrication -----
43 quant, bins = quantize(phi, n_levels=8)
44 entry = FDTDStore().get(532)
45 info = GDSEngine().generate(
```

```

46     quantized_mask=quant, bins=bins,
47     dimensions=entry.dimensions, phases=entry.phases,
48     period=entry.period, pixel_size_um=0.5, unitcells_per_pixel=1,
49     shape=entry.shape, is_circular=True,
50     output_path="outputs/metalens_532nm.gds",
51     fin_width=entry.fin_width or 0, fin_length=entry.fin_length or 0,
52 )
53 print(f"GDS written: {info['filepath']} ({info['file_size_kb']:.1f} kB)")

```

Listing A.1: Complete design flow: spec  $\rightarrow$  RCWA  $\rightarrow$  phase  $\rightarrow$  analysis  $\rightarrow$  GDS.

## Appendix B

# Legacy AI Layer (`old_one/`)

An earlier iteration of METAOPTICSAI shipped seven additional subpackages that layered a Self-RAG LangGraph workflow, an LLM provider abstraction and an MCP (Model Context Protocol) server tree on top of the physics core. These are preserved for reference under `old_one/`:

- `old_one/llm/` — pluggable LLM providers (Ollama, Groq, Anthropic).
- `old_one/rag/` — FAISS-backed knowledge base with a document grader.
- `old_one/mcp_servers/` — MCP servers exposing the tools/ registry over stdio or HTTP.
- `old_one/orchestration/` — controller and job manager.
- `old_one/prompts/` — design / reflection / summarize prompt templates.
- `old_one/store/` — concrete `ArtifactStore` implementation.
- `old_one/workflows/` — LangGraph node definitions for the Self-RAG AutoML flow.
- `old_one/notebooks/` — notebooks demonstrating the above (04–06).

These directories are no longer imported by the physics core and are not maintained. Nothing in the current `mod src/metaopticsai/dependsonthem`.

# Bibliography

- [1] N. Yu *et al.*, “Light Propagation with Phase Discontinuities: Generalized Laws of Reflection and Refraction,” *Science* **334**, 333–337 (2011).
- [2] M. Khorasaninejad *et al.*, “Metalenses at visible wavelengths: Diffraction-limited focusing and subwavelength resolution imaging,” *Science* **352**, 1190–1194 (2016).
- [3] F. Capasso, “The future and promise of flat optics: a personal perspective,” *Nanophotonics* **7**(6), 953–957 (2018).
- [4] M. G. Moharam *et al.*, “Formulation for stable and efficient implementation of the rigorous coupled-wave analysis of binary gratings,” *J. Opt. Soc. Am. A* **12**(5), 1068–1076 (1995).
- [5] L. Li, “Fourier Modal Method,” in *Gratings: Theory and Numeric Applications* (E. Popov, ed.), 2013.
- [6] L. Li, “Use of Fourier series in the analysis of discontinuous periodic structures,” *J. Opt. Soc. Am. A* **13**, 1870–1876 (1996).
- [7] R. Redheffer, “Difference equations and functional equations in transmission-line theory,” in *Modern Mathematics for the Engineer*, McGraw-Hill (1961).
- [8] G. J. Gbur, *Mathematical Methods for Optical Physics and Engineering*, Cambridge University Press (2011).
- [9] J. W. Goodman, *Introduction to Fourier Optics*, 3rd ed., Roberts & Company (2005).
- [10] M. Born and E. Wolf, *Principles of Optics*, 7th ed., Cambridge University Press (1999).
- [11] G. J. Swanson, “Binary optics technology: theoretical limits on the diffraction efficiency of multilevel diffractive optical elements,” MIT Lincoln Lab Tech. Rep. 914 (1991).
- [12] R. J. Noll, “Zernike polynomials and atmospheric turbulence,” *J. Opt. Soc. Am.* **66**(3), 207–211 (1976).